

Elevator Term Project

Correctness By Construction

Christopher Roßbach David Hospodka Antonio José Fernández Pinto

Universidad Politécnica de Madrid

21 May 2025

1 Base Model: Machine 0

- Model Diagram
- Invariants for Requirements

2 Refinements

- The Need for Refinements
- The Algorithm
- Informal Argument for the Liveness of the Algorithm
- Refinements Diagram
- Realization in Rodin
- Deadlock Freedom
- State Transitions

3 Appendix

- Modelling Environment
 - Reading from the Environment
 - Controlling the Environment
- Possible Improvements
 - Door Modelling
 - Reduce Direction Events

Model Diagram

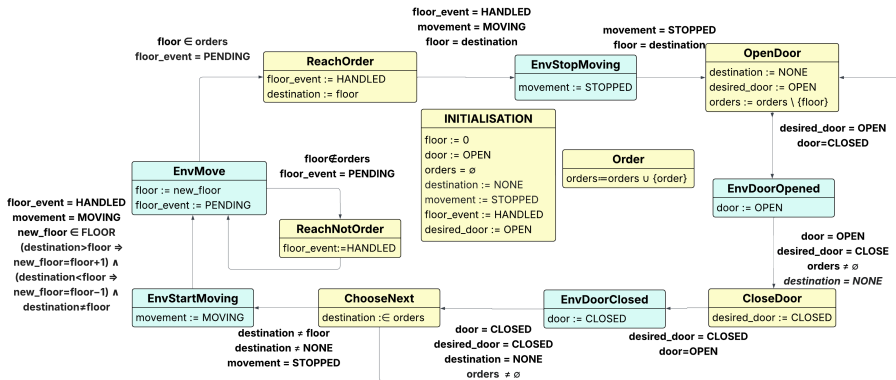


Figure: Diagram of the model

Invariants for Requirements: Elevator Movement

- | | |
|---------------|--|
| FUN 16 | The elevator should not move in a direction where there are no pending requests to attend. |
|---------------|--|

`inv_move_to_ordered: destination  $\neq$  NONE  $\implies$  destination  $\in$  orders`

- | | |
|---------------|---|
| FUN 15 | The elevator should not move if there are no requests waiting to be served. |
|---------------|---|

`inv_only_move_with_ordersTHM: movement=MOVING  $\implies$  orders  $\neq$   $\emptyset$`

This follows directly from **FUN 16** and the helper:

`inv_only_move_when_destination: movement=MOVING  $\implies$  destination  $\neq$  NONE`

Invariants for Requirements: Door Management

- Necessary to ensure movement restrictions according to door status

door	desired_door	Meaning
OPEN	OPEN	The door is open and stays open
OPEN	CLOSED	The door is closing
CLOSED	OPEN	The door is opening
CLOSED	CLOSED	The door is closed and stays closed

- **FUN 8** The elevator doors should be closed when the elevator is moving.

$\text{inv_door_closed_when_moving}^{\text{THM}}:$

$\text{movement}=\text{MOVING} \implies \text{door}=\text{CLOSED} \wedge \text{desired_door}=\text{CLOSED}$

Helper:

$\text{inv_door_closed_when_destination}:\text{destination} \neq \text{NONE} \implies \text{door}=\text{CLOSED}$

$\text{inv_no_dest_when_opening_door}:\text{desired_door}=\text{OPEN} \implies \text{destination}=\text{NONE}$

- **FUN 11** The doors should be open if there are no requests to serve.

$\text{inv_no_orders_door_open}^{\text{THM}}:\text{orders}=\emptyset \implies \text{door}=\text{OPEN} \vee \text{desired_door}=\text{OPEN}$

Helper:

$\text{inv_orders_when_closing_door}:\text{desired_door}=\text{CLOSED} \implies \text{orders} \neq \emptyset$

The Need for Refinements: Deterministic Serving Order

- So far: **ChooseNext** selects `destination: ∈ orders` randomly

FUN 13	All pending requests should be eventually be served.
---------------	--

- Imagine this situation:

floor	orders	Description
0	{1,4}	We are at the ground floor. Floors 1 and 4 are ordered.
1	{4}	We select floor 1 and served it.
1	{0,4}	Somebody ordered the ground floor.
0	{4}	We select floor 0 and served it.
0	{1,4}	Somebody ordered floor 1. This can repeat infinitely.

The Algorithm

- We'll just use the elevator algorithm.
- **ChooseNext** gets transformed into four new events.
 - ▶ **ChangeDirectionUp**
 - ▶ **ChangeDirectionDown**
 - ▶ **ChooseNextUp**
 - ▶ **ChooseNextDown**

Informal Argument for the Liveness of the Algorithm

- Properties this algorithm has:
 - ▶ The elevator moves/starts moving when there are any orders.
 - ▶ The elevator stops at every floor that has been ordered and that is in its immediate path.
 - ▶ If the elevator is going up/down, then it continues to do so as long as there are orders above/below.
 - ▶ When it runs out of orders above/below, but it still has orders the other way, it changes direction.
- Then, for any floor that is ordered:
 - ▶ The elevator may be at that floor already, so it is served.
 - ▶ The elevator is not currently at that floor. Then either:
 - ★ It's moving in the opposite direction, in which case it will eventually turn around.
 - ★ It's moving in the correct direction, in which case it will eventually reach the ordered floor and serve it.

Refinements Diagram

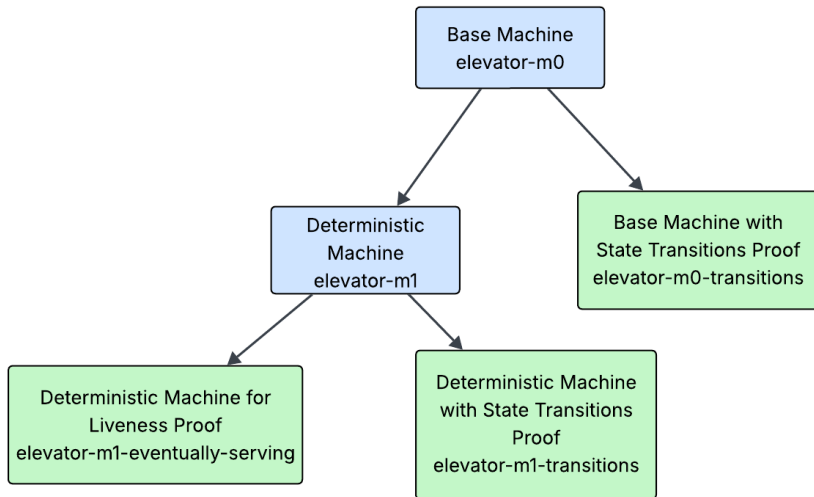


Figure: Refinements family tree

Realization in Rodin

- Add a refinement to isolate everything cleanly
- Define a variable `waiting_since` that tracks the waiting time for each order:
`waiting_type:waiting_since∈FLOOR→ℕ`
- Assume an upper limit on the waiting time using an invariant
`waiting_limit:∀x·x∈FLOOR⇒waiting_since(x)<N_Levels*2`
- In **OpenDoor** add the Action
`act_inc_and_reset:waiting_since:=
((waiting_since;add_one)⊧{floor↦0})⊧((FLOOR\orders)×{0})`
- Where `add_one` is defined as a constant in the context with the axioms
 - ▶ `add_one∈ℕ→ℕ1`
 - ▶ `∀n·n∈ℕ⇒add_one(n)=n+1`
- We were not able to proof this invariant in Rodin
- We did ProB model checking to search for invariant violations
 - ▶ Check with different numbers of floors `N_Levels∈{1,2,3,4,5}`
 - ▶ No invariant violations found
 - ▶ Mark as reviewed
 - ▶ Workaround for ProB `add_one:add_one:={0↦1, 1↦2, 2↦3, ...}`

Deadlock Freedom

- When there are orders, we want to ensure there is always a possible event, that is not **Order**.
- $\text{inv_no_deadlock}^{\text{THM}} : \text{orders} \neq \emptyset \implies (\text{guards for } \text{EnvMove}) \vee (\text{guards for } \text{ReachOrder}) \vee (\text{guards for } \text{ReachNotOrder}) \vee \dots$

State Transitions

- The guards for the events are fairly complicated, so it's not obvious they follow each other like the arrows of the shown diagrams. For instance, does **EnvStopMoving** always come after, and only after, **ReachOrder**.
- To prove this, we can make a refinement that doesn't change the behaviour of the base model, but adds a state variable that is set to a different value for every possible state in between events.
- Only **ReachOrder** sets state to 6, and
`inv6:state = 6  $\Leftrightarrow$  (guards for EnvStopMoving).`

Thank you for your attention!

Reading from the Environment

- The controller is able to read the environment state directly

Variable	Value	Description
movement	MOVING	The elevator is (physically) moving
	STOPPED	The elevator is (physically) not moving
door	OPEN	The elevator door is (physically) open [†]
	CLOSED	The elevator door is (physically) closed [†]
floor	0..N_Levels	The floor the elevator is (physically) on [‡]

[†] the value changes as soon as the door is fully open/closed

[‡] the next possible floor to stop

- The controller can read and set the order button state
 - We consider changes to this variable to be instantaneous

FUN 12	The status of the buttons can be accessed from the controller.	
Variable	Value	Description
orders	$\subseteq$ floor	The set of buttons where the state is ON

Controlling the Environment

- The controller can communicate with the environment by giving commands

FUN 6	The elevator can receive a signal to move to any level and will stop automatically when it is reached [...].
--------------	--

FUN 7	The elevator has doors that can be closed or opened from a controller.
--------------	--

Variable	Value	Description
destination	$\in 0..N_Levels$	The elevator shall move to floor
	NONE	The elevator shall not move
desired_door	OPEN	The elevator shall open the door
	CLOSED	The elevator shall close the door

- Stopping $\implies$ this is handled by `destination=floor`

Possible Improvements: Door Modelling

- The modelling of the physical door state is not completely "clean"
 - ▶ E.g. the door can be CLOSED while the door is currently opening
- Options to fix this came to our minds:
 - ➊ Add additional physical states OPENING and CLOSING
 - ★ New environment events **EnvStartOpening** and **StartClosing** would be needed to adapt the physical state when $\text{door} \neq \text{desired_door}$
 - ★ This adds complexity to the model
 - ★ It contradicts the idea, that $\text{desired_door} = \text{OPEN}$ instantly triggers the opening
 - ➋ Add a function $\text{physical_door_state} : (\text{DOOR} \times \text{DOOR}) \rightarrow \{\text{OPEN}, \text{OPENING}, \text{CLOSED}, \text{CLOSING}\}$, that interprets door and desired_door
 - ★ Usage would change for example the invariant $\text{inv_no_orders_door_open}$ which was $\text{orders} = \emptyset \implies \text{door} = \text{OPEN} \vee \text{desired_door} = \text{OPEN}$ to $\text{orders} = \emptyset \implies (\text{physical_door_state}(\text{door}, \text{desired_door}) \in \{\text{OPEN}, \text{OPENING}\})$
 - ★ This can improve readability
 - ★ But it increases proof complexity
 - ➌ Use a single variable for door which is one of OPEN, OPENING, CLOSED, CLOSING
 - ★ Setting $\text{door} := \text{CLOSING}$ by the controller would make the door start closing
 - ★ The environment would set this variable upon finishing to $\text{door} := \text{CLOSED}$
 - ★ This contradicts the idea, that the variables are only changed by the either environment or the controller
 - ★ This exception may be acceptable since orders behaves in a similar way

Reduce Direction Events

- We can define a helper function $\text{cut}(\text{floor}, \text{orders}, \text{direction})$ that returns the remaining orders in the direction of the current movement
- This functions has the axioms:

- ▶ $\text{axm_cut_type: cut} \in (\text{FLOOR} \times \mathbb{P}(\text{FLOOR}) \times \text{DIRECTION}) \implies \mathbb{P}(\text{FLOOR})$

- ▶ axm_cut_up:

- $\forall \text{floor}, \text{orders}, \text{direction}.$

- $\text{floor} \in \text{FLOOR} \wedge \text{orders} \subseteq \text{FLOOR} \wedge \text{direction} \in \text{DIRECTION} \wedge \text{direction} = \text{UP}$
 $\implies \text{cut}(\text{floor} \mapsto \text{orders} \mapsto \text{direction}) = \{x \mid x \in \text{orders} \wedge x > \text{floor}\}$

- ▶ axm_cut_down:

- $\forall \text{floor}, \text{orders}, \text{direction}.$

- $\text{floor} \in \text{FLOOR} \wedge \text{orders} \subseteq \text{FLOOR} \wedge \text{direction} \in \text{DIRECTION} \wedge \text{direction} = \text{DOWN}$
 $\implies \text{cut}(\text{floor} \mapsto \text{orders} \mapsto \text{direction}) = \{x \mid x \in \text{orders} \wedge x < \text{floor}\}$

Reduce Direction Events

- This would lead to reduction in events and avoids copy-paste-errors:
 - ▶ events **ChangeDirectionUp** and **ChangeDirectionDown** can be unified
 - ★ Common guards and actions stay
 - ★ The guard $\text{cut}(\text{floor} \mapsto \text{orders} \mapsto \text{direction}) = \emptyset$
 - ★ and the action $\text{direction} : \in \text{DIRECTION} \setminus \{\text{direction}\}$ are added
 - ▶ events **ChooseNextUp** and **ChooseNextDown** can be unified
 - ★ Common guards and actions stay
 - ★ The guard $\text{cut}(\text{floor} \mapsto \text{orders} \mapsto \text{direction}) \neq \emptyset$
 - ★ and the action $\text{direction} : \in \text{cut}(\text{floor} \mapsto \text{orders} \mapsto \text{direction})$
 - ★ To choose deterministic max or min could be used but this is not necessary
 - ★ Even a function like `next_cut` that chooses the max or min of the cut-set depending on the direction is possible
- We implemented this but a lot of proofs required repeated manual intervention as we made changes to the model
- Using the separate events made the proof obligations discharge automatically which was a big help in the iterative development, so we chose the separate event approach